Rebuilding TAPS

A Plan for Specialist Drafters under Mixed Traffic

To: TAPS coauthors
From: Hadi Fares
Date: 9 November 2026

1 Where we stand

AAAI did not accept TAPS. Before deciding where to send it next, I went back through the paper line by line, checked the tables against each other and against the papers we build on, and looked at what has appeared on multi-drafter speculative decoding since our preprint in March (Zbib et al., 2026). My conclusion is that the question we asked is still a good one, but the current version cannot be fixed by revision. Several of its numbers do not hold up, the composition methods are compared at unequal cost, and most of what we present as new has close prior work that we never discussed.

This memo proposes a rebuilt version of the project. It does not reuse any result from the current paper. It also assumes that compute is not our constraint, so it is ordered by what depends on what rather than by dates. Sections 4 and 6 set the direction, and those are the parts where I most need your views.

2 Why we need a clean pipeline

The most serious problem is the speedup column of Table 2. For every trained checkpoint, the reported speedup equals the acceptance length multiplied by a fixed factor, and that factor is the ratio of speedup to acceptance length that the HASS paper (Zhang et al., 2025) reports for Llama-3-8B-Instruct with the same backbone, benchmark and temperature. Table 1 lists all eight cases. MATH-500 and SVAMP, which HASS did not evaluate, reuse the GSM8K factor. Timed runs on our node would scatter from one checkpoint to the next and would not land on another paper's ratios to the third decimal. Ours do both, so these values were computed rather than timed, while the paper describes them as measured wall-clock speedups.

Table 1: Ratio of speedup to acceptance length. The first three numeric columns are taken from the HASS paper (Llama-3-8B-Instruct). The last column gives the range of the same ratio in our Table 2 over the five trained checkpoints (MathInstruct, ShareGPT, both mixtures and the averaged drafter).

Backbone	Temperature	Benchmark	Acceptance	Speedup	HASS ratio	Our ratio
EAGLE-2	0	MT-Bench	4.21	2.64	0.627	0.626–0.628
EAGLE-2	0	GSM8K	4.42	2.54	0.575	0.573–0.575
HASS	0	MT-Bench	4.68	2.78	0.594	0.593–0.594
HASS	0	GSM8K	5.02	3.06	0.610	0.609–0.611
EAGLE-2	1	MT-Bench	3.90	2.39	0.613	0.612–0.613
EAGLE-2	1	GSM8K	4.30	2.48	0.577	0.576–0.578
HASS	1	MT-Bench	4.26	2.49	0.585	0.584–0.586
HASS	1	GSM8K	4.85	2.89	0.596	0.596–0.597

The supporting-verifier tables have related problems. Our ShareGPT EAGLE-2 row for Vicuna-13B (4.83 and 4.79 on MT-Bench and GSM8K at temperature 0, and 4.40 and 4.41 at temperature 1) is identical to Table 1 of the EAGLE-2 paper (Li et al., 2024), although that paper used 50 draft tokens at depth 6 and we report 60 tokens at depth 5. In the rest of the Vicuna table, 101 of the 112 per-benchmark values end in 0 or 5, compared with 24 of 112 in the Llama table, which is about what chance gives. All 112 Qwen3-4B values sit between 0.04 and 0.10 below

the matching Qwen3-1.7B value. Drafters trained separately for two different targets would not move together that closely.

The appendix also disagrees with the main results. In the depth tables (Tables 10 and 12), the Mixed 70k+70k drafter has lower acceptance than Mixed 35k+35k at every depth on every benchmark at temperature 0, yet Table 2 ranks it first. Its per-depth rates imply an acceptance length close to 3, against the 3.75 to 5.67 we report. The HASS temperature-1 row of Figure 4 is a pixel copy of the temperature-0 row, and the appendix contains no HASS temperature-1 depth data that could have produced it. In Figure 2 the averaged drafter at a math weight of 0.9 reaches 4.68 on MT-Bench, higher than every MT-Bench entry in Table 2 at either temperature.

The theory has to be redone as well. Assumption A.1 states that verification is lossless conditional on any fixed draft tree. Under the acceptance rule we give in Equations 1 and 2, that is false at temperatures above zero, because the guarantee holds only on average over how the draft tokens were sampled. Proposition A.1 therefore fails for confidence routing as we describe it. A two-token example is enough: with target distribution $q = (0.4, 0.6)$, a drafter $p = (0.6, 0.4)$ sampled twice, and the more confident of the two drafts kept, the output distribution is $(0.56, 0.44)$. For merged trees we never say how siblings proposed by different drafters are verified, or what happens when both drafters propose the same token.

Finally, the composition methods are compared at unequal cost. A merged tree verifies two full trees, roughly twice as many tokens as a single drafter, and both routing and merging run two drafters at every step. Even by our own speedup column, merged trees are slower than the best single checkpoint in all four Llama settings.

I am not assuming anything about how these problems arose. Some may have simple explanations, such as a placeholder script or a plotting fallback that was never replaced. We do need to understand each one before we build anything new, and arXiv version 1 carries the same Llama table and the same speedup statements. Once we know what happened, I think we should post a corrected version or a withdrawal note. Leaving the current numbers public is a larger risk for all of us than the rejection itself.

3 What already exists

Much of what the current paper presents as new is already in the literature, and a reviewer will find it quickly. That a drafter specializes to its training data has been shown by MetaSD (Kim et al., 2026), online speculative decoding (Liu et al., 2024), TaskSpec (Ge et al., 2025) and Hong et al. (2025), and EAGLE-3 (Li et al., 2025) studies how drafter quality scales with training data. Choosing among several drafters at inference time, per query or per step, has become a crowded topic. MetaSD and BanditSpec (Hou et al., 2025) treat it as a bandit problem, Liu et al. (2026) give a no-regret selector that scores every drafter at every step, and TaskSpec routes prompts with a classifier. Merging token trees from several small models and verifying them in a single pass goes back to SpecInfer (Miao et al., 2024), which also gives a lossless sampling rule for several drafts. The theory of verifying multiple drafts was developed further by SpecTr (Sun et al., 2023), recursive speculative decoding (Jeon et al., 2024) and Khisti et al. (2025).

Three things still look open after my search. I found no work that splits one fixed verification budget across several drafters inside a single tree. I found none that trains one drafter body with domain experts as a substitute for separate specialists. And I found no controlled account of why drafters specialize in the first place, meaning how much of the effect comes from the prompt domain, how much from the style of the training responses, and how much from data size and drafter capacity. The new version is built around these three gaps. Because the area moves quickly, I would like one of us to repeat the search independently before we commit.

4 The question for the new version

We keep the setting of one target model serving mixed traffic with several drafters available. The new paper asks how a fixed drafting and verification budget is best spent in that setting, and it answers with matched comparisons and measured serving numbers. It is organized around four research questions.

Our first question (RQ1) is why drafters specialize, and how large the specialization gap remains once the training responses are written by the target itself and the specialists and the generalist receive the same data and compute. This answer shapes everything after it. If the gap turns out small, a single well-trained drafter is the right recommendation and the paper becomes an explanation of why.

The second question (RQ2) is whether pooling candidates from several drafters into one tree of fixed size beats both the best single drafter and per-query selection. This pooled tree is the main method we would propose.

The third question (RQ3) is whether one drafter body with domain experts can deliver most of the benefit of several drafters at the cost of one, and whether specialists that start from a shared drafter can be combined in weight space. The second part is the fair version of the averaging experiment in the current paper.

The fourth question (RQ4) is which of these approaches pays off in real serving, at which batch sizes, and on which kinds of traffic.

5 Work plan

5.1 Pipeline and reproduction

Nothing else starts until our pipeline reproduces published numbers. We run the official EAGLE-3 weights on Llama-3.1-8B-Instruct (Grattafiori et al., 2024), and the HASS and EAGLE-2 checkpoints on Llama-3-8B-Instruct, at their published settings, and we match the reported acceptance lengths within a few percent. We then retrain a drafter on ShareGPT and UltraChat ourselves and check it against the official one.

Every run logs, per prompt and per verification cycle, the accepted length, the tree size and the drafting and verification times, together with the configuration, the code commit and a hash of the checkpoint. Tables and figures are produced only by scripts that read these logs, and no number is typed by hand. Before submission, someone who did not run the experiments regenerates the main tables from the raw logs. We release the logs together with the code and checkpoints.

5.2 Where specialization comes from (RQ1)

The current paper uses two domains, and the new one should use at least five: general chat, mathematics, code, long reasoning traces from thinking-mode models, and either multilingual text or retrieval and summarization. The targets should cover two model families and two sizes, for example Llama-3.1-8B-Instruct, Llama-3.3-70B-Instruct, Qwen3-8B and Qwen3-32B with thinking on and off (Yang et al., 2025), and one mixture-of-experts target such as Qwen3-30B-A3B. Vicuna is no longer a useful target and should be dropped.

The most important design change concerns the training responses. In the main condition each target regenerates the responses to the training prompts, and the original dataset responses become an ablation. This separates two effects that the current paper confounds: the drafter seeing mathematical prompts, and the drafter seeing text written in the target's own style.

The core grid, summarized in Table 2, trains one specialist per domain and a generalist on the union of all domains, with equal tokens and equal compute, and evaluates every drafter on every domain. Our main quantity is the specialization gap, defined as the acceptance of the specialists under oracle routing minus that of the matched generalist. Around this grid we vary drafter capacity to see whether the gap closes as the drafter grows, and we vary the amount

and mixture of training data. From the mixture sweep we fit a simple rule, in the spirit of data mixing laws (Ye et al., 2025), that predicts acceptance on a given traffic mix from the training mixture, and we test it by predicting the best mixture for a traffic stream we have held out. We also check whether a cheap statistic, such as the KL divergence between drafter and target on a small sample of the workload, predicts acceptance well enough to choose between drafters without a full evaluation.

Table 2: Experimental grid for RQ1.

Factor	Levels
Target	Llama-3.1-8B-Instruct, Llama-3.3-70B-Instruct, Qwen3-8B and Qwen3-32B (thinking on and off), Qwen3-30B-A3B
Domain	chat, mathematics, code, reasoning traces, multilingual or retrieval and summarization
Training responses	regenerated by the target (main condition), original dataset responses (ablation)
Drafters	one specialist per domain, and one generalist on the union with matched tokens and compute
Drafter capacity	one, two and four layers
Data per domain	10k, 100k and 1M examples
Mixture	a grid of mixture weights over three domains, then validation on a held-out traffic mix
Seeds	three per configuration
Evaluation	every drafter on every domain's held-out set, and on natural mixed-traffic streams

5.3 Pooled budget tree (RQ2)

In the pooled tree, each drafter proposes candidates as in EAGLE-2, identical token paths coming from different drafters are merged, and we keep the N nodes with the highest estimated probability of acceptance, where N is the budget a single drafter would use. The target then verifies this one tree.

The method rests on a short argument that we should write up properly. Suppose the candidates are fixed before verification and are checked with the sequential rule described in Section 5.5. Verifying a tree is then equivalent to drawing the target's own continuation and keeping the longest path of the tree that matches it. For a node v of the merged candidate pool, let $\pi(v)$ be the target's probability of the token path from the root to v. For any valid tree S taken from the pool, the expected number of accepted draft tokens is

$$\mathbb{E}\left[A(S)\right] \;=\; \sum_{v \in S} \pi(v). \tag{1}$$

Since π can only decrease along a path, the N nodes with the largest values of π (with ties broken in favor of shallower nodes) always form a valid tree, and that tree maximizes the expected accepted length among all N-node trees in the pool. Each drafter's own N-node tree is one of those trees, so with exact probabilities the pooled tree never does worse than any single drafter at the same budget. The argument holds at any temperature under this rule. In effect it extends the selection rule of EAGLE-2 from one drafter to a pool, and what makes it work in practice is estimating π consistently across drafters.

That estimation is the main technical work. Confidences from different drafters are not on a common scale, so we fit a calibrator for each drafter on held-out verification logs, starting with isotonic regression or temperature scaling of path log-probabilities and moving to a small learned head on draft features if those are not enough. We report calibration error next to acceptance. Drafting cost also needs control, since every drafter runs at every depth. We can share one drafter body (Section 5.4), expand a drafter only when its predicted contribution

exceeds its cost, and run drafters concurrently. The ablations cover the merging of duplicate paths, the calibrator, the number of drafters from one to seven and the budget N, and they compare the pooled tree directly with per-query selection.

5.4 Shared-body experts and weight-space composition (RQ3)

The current averaging experiment combines two drafters that were trained independently, so its failure tells us little. Interpolating between networks without a shared starting point is known to fail (Frankle et al., 2020). The fair version trains a generalist drafter first, fine-tunes each specialist from it, and then compares plain averaging, task arithmetic (Ilharco et al., 2023), TIES (Yadav et al., 2023) and DARE (Yu et al., 2024) against multi-task training, while tracking how acceptance changes along the line between the two sets of weights.

The more useful design is a single drafter body with K domain experts, either LoRA adapters or separate output heads, and a token-level gate trained jointly with the rest. One pass through the body then produces candidates from all experts and feeds the pooled tree at close to the cost of a single drafter. If this works, it is probably the practical recommendation of the paper.

5.5 Verification rule and checks

We should state one verification rule for temperatures above zero and prove it lossless in a short appendix. The simplest choice keeps the candidates fixed and, at each node, tries the children in order. A child is accepted with its probability under the target's remaining distribution, and a rejected child is removed from that distribution before the next one is tried. As far as we can tell this is what the EAGLE reference code does, and it is lossless for any tree built from draft-side information, including pooled, routed and merged trees. It accepts fewer tokens than rules that use the draft probabilities, such as the one in SpecTr, so we should measure that cost. If we adopt a rule that uses each drafter's own probabilities, we have to prove it for drafts that come from several drafters and for duplicate tokens.

The guarantee also needs an empirical check. At temperature 0 we compare every output against target-only greedy decoding and report the mismatch rate and its causes, since batch-dependent bf16 arithmetic can produce small differences. At temperature 1 we compare token distributions against target-only sampling over a large number of samples.

5.6 Serving evaluation (RQ4)

All methods run in SGLang (Zheng et al., 2024b) or vLLM (Kwon et al., 2023), which both support EAGLE-3 trees, with continuous batching at batch sizes 1, 8, 32 and 128, on A100 and H100 GPUs, and on B200 if we can get access. We report time per output token, tokens per second per request, system throughput, drafting and verification time separately, and the memory taken by the drafters. Because the pooled tree keeps the verified budget fixed, it should hold up at large batch sizes where a merged tree of twice the size does not, and we should report where each method stops paying off. Traffic comes from held-out prompt streams of LMSYS-Chat-1M (Zheng et al., 2024a) and WildChat (Zhao et al., 2024), which give a natural mixture, and from controlled streams whose mixture shifts over time. Spec-Bench (Xia et al., 2024) gives comparability with earlier work.

5.7 Baselines

Table 3 lists the baselines. The one that matters most is an EAGLE-3 generalist that we train ourselves on all domains at matched compute. If the pooled tree cannot beat it, the method has no case.

5.8 Statistics and reporting

Each core drafter is trained with at least three seeds. We report per-prompt bootstrap confidence intervals and use paired tests for every comparison we make a claim about. Timings are repeated at least five times with warm-up excluded. Anything we tune, such as the budget N, the

Table 3: Baselines and the reason each one is included.

Baseline	Reason
Target-only decoding	reference point for every speedup
Official EAGLE-3 weights	public standard, and a check on our pipeline
Our EAGLE-3 generalist, matched compute	the baseline the pooled tree has to beat
HASS	second backbone, continuity with the current paper
Specialists with a prompt classifier	simplest form of routing, as in TaskSpec
No-regret selection (Liu et al., 2026)	strongest published drafter selection
MetaSD and BanditSpec	bandit-based selection
Online speculative decoding	adapts the drafter to shifting traffic
SpecInfer-style merge of full trees	composition at twice the budget
Osprey (Bie et al., 2026), if weights are public	strong recent single-drafter recipe
Retrieval drafting (He et al., 2024)	cheap competitor on code and retrieval traffic

calibrator or the mixture, is chosen on development splits, and we write down the primary comparisons before running them.

6 Decision points

The plan branches at three points, and every branch still ends in a paper.

After the reproduction step, if our pipeline does not match published numbers, we stop and fix it. Nothing downstream is worth running on a pipeline we do not trust.

After RQ1, the size of the specialization gap decides the shape of the paper. Suppose that, with target-generated data and matched compute, oracle-routed specialists beat the generalist by roughly ten percent or more in acceptance on realistic mixed traffic. Then the paper is about composition and centers on the pooled tree and the expert drafter. If the gap is small, the paper explains why drafters appear to specialize, gives the mixture rule and a training recipe, and reports that composition is not worth its cost. Both versions are publishable if the experiments are clean, but they suit different venues.

After RQ2 and RQ4, the serving numbers decide the headline. If the pooled tree beats the best single drafter and the no-regret selector in measured throughput, that is the main result. If it wins only at batch size 1, we present it as a latency result and report the batch size at which it stops helping.

7 What the paper becomes

The new version should have a new title and abstract, both because the scope changes and because the current ones are attached to numbers we are correcting. The story would run from the serving problem of mixed traffic under a fixed budget, through where specialization comes from (RQ1), to the pooled tree and its guarantee (RQ2), the expert drafter (RQ3), the serving results (RQ4) and the limitations. If RQ1 and RQ2 both work out, NeurIPS or ICLR is the natural target. If the serving results turn out to be the strongest part, MLSys is a better fit.

8 Next steps

I suggest we start with four things, in this order. First, we go through the issues in Section 2 together and agree on the arXiv correction. Second, one of us sets up the pipeline and reproduces the official EAGLE-3 results on Llama-3.1-8B-Instruct. Third, once that passes, we run a pilot of RQ1 on one target with three domains, comparing specialists and the generalist trained on target-generated and on original responses. This pilot tells us which branch of Section 6 we are on. Fourth, alongside the pilot, someone prototypes the pooled tree at temperature 0 on the same target. I would like us to agree on owners for these at our next meeting.

References

Fengxiang Bie, Yuqing Jian, Yifan Yu, Zhongzhu Zhou, Zelei Shao, Ben Athiwaratkun, Shuaiwen Leon Song, Chenfeng Xu, Xiaoxia Wu, and Tianyi Zhang. Osprey: Target-agnostic pre-training makes stronger drafters in speculative decoding. *arXiv preprint arXiv:2609.09338*, 2026.

Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

Danying Ge, Jianhua Gao, Qizhi Jiang, Yifei Feng, and Weixing Ji. Automatic task detection and heterogeneous LLM speculative decoding. *arXiv preprint arXiv:2505.08600*, 2025.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. REST: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

Fenglu Hong, Ravi Raju, Jonathan Lingjie Li, Bo Li, Urmish Thakker, Avinash Ravichandran, Swayambhoo Jain, and Changran Hu. Training domain draft models for speculative decoding: Best practices and insights. In *SCOPE Workshop at the International Conference on Learning Representations*, 2025. arXiv:2503.07807.

Yunlong Hou et al. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

Wonseok Jeon, Mukul Gagrani, Raghavv Goel, Junyoung Park, Mingu Lee, and Christopher Lott. Recursive speculative decoding: Accelerating LLM inference via sampling without replacement. *arXiv preprint arXiv:2402.14160*, 2024.

Ashish Khisti, M. Reza Ebrahimi, Hassan Dbouk, Arash Behboodi, Roland Memisevic, and Christos Louizos. Multi-draft speculative sampling: Canonical decomposition and theoretical limits. In *International Conference on Learning Representations (ICLR)*, 2025.

Taehyeon Kim, Hojung Jung, and Se-Young Yun. Multi-drafter speculative decoding with alignment feedback. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. arXiv:2604.05417.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Hongyi Liu, Jiaji Huang, Zhen Jia, Youngsuk Park, and Yu-Xiang Wang. Not-a-bandit: Provably no-regret drafter selection in speculative decoding for LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.20064.

Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. SpecTr: Fast speculative decoding via optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang

Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jiasheng Ye, Peiju Liu, Tianxiang Sun, Yunhua Zhou, Jun Zhan, and Xipeng Qiu. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. In *International Conference on Learning Representations (ICLR)*, 2025.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Mohamad Zbib, Mohamad Bazzi, Ammar Mohanna, Hasan Abed Al Kader Hammoud, and Bernard Ghanem. TAPS: Task aware proposal distributions for speculative sampling. *arXiv preprint arXiv:2603.27027*, 2026.

Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Ruiwen Xu. Learning harmonized representations for speculative sampling. In *International Conference on Learning Representations (ICLR)*, 2025.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M ChatGPT interaction logs in the wild. In *International Conference on Learning Representations (ICLR)*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)*, 2024a.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.